



PDF Download
3699965.pdf
15 February 2026
Total Citations: 1
Total Downloads: 593

Latest updates: <https://dl.acm.org/doi/10.1145/3699965>

RESEARCH-ARTICLE

ConsortiumSec: Blockchain-Based Distributed Security Framework for Consortium Applications

KEWEI SHA, University of North Texas, Denton, TX, United States

KWOK BUN YUE, University of Houston-Clear Lake, Houston, TX, United States

WEI WEI, University of Houston-Downtown, Houston, TX, United States

YALONG WU, University of Houston-Clear Lake, Houston, TX, United States

MADHURI KODURU, University of Houston-Clear Lake, Houston, TX, United States

PREETHI VUCHURU, University of Houston-Clear Lake, Houston, TX, United States

Open Access Support provided by:

University of Houston-Downtown

University of North Texas

University of Houston-Clear Lake

Published: 08 October 2025
Online AM: 10 October 2024
Accepted: 25 September 2024
Revised: 28 August 2024
Received: 25 October 2023

[Citation in BibTeX format](#)

ConsortiumSec: Blockchain-Based Distributed Security Framework for Consortium Applications

KEWEI SHA, University of North Texas, Denton, TX, USA

KWOK-BUN YUE, University of Houston-Clear Lake, Houston, TX, USA

WEI WEI, University of Houston-Downtown, Houston, TX, USA

YALONG WU, MADHURI KODURU, and PREETHI VUCHURU, University of Houston-Clear Lake, Houston, TX, USA

In consortium applications, multiple organizations collaborate on a common goal. The complex consortium structure makes it a significant challenge to achieve the goals of both secure collaboration and preserving each organization's data confidentiality. Existing security architectures are insufficient to tackle the above challenge. This article proposes ConsortiumSec, a blockchain-based two-layer distributed security architecture. ConsortiumSec leverages disruptive blockchain technologies to systematically address the security issues in consortium applications, including consortium membership management, access control, secure storage, governance policy management, and privacy preservation. The Hyperledger Fabric-based prototype implementation demonstrates the proposed architecture not only integrates the organization's existing security mechanisms but also ensures a secure collaborative environment. The case study based on a real-world consortium application demonstrates the effectiveness of ConsortiumSec.

CCS Concepts: • **Security and privacy** → **Domain-specific security and privacy architectures**; • **Computing methodologies** → **Self-organization**;

Additional Key Words and Phrases: Blockchain, security, privacy, architecture, consortium, distributed, multi-organization

ACM Reference format:

Kewei Sha, Kwok-Bun Yue, Wei Wei, Yalong Wu, Madhuri Koduru, and Preethi Vuchuru. 2025. ConsortiumSec: Blockchain-Based Distributed Security Framework for Consortium Applications. *Distrib. Ledger Technol.* 4, 4, Article 28 (October 2025), 19 pages.

<https://doi.org/10.1145/3699965>

1 Introduction

Collaboration is critical in modern society as complex projects require diverse types of expertise from many collaborators. A consortium always consists of a group of organizations/members that work together on a

This article was partially supported by the National Aeronautics and Space Administration (NASA) STTR program under Grant No. 80NSSC21C0020 and NSF Award #2240513.

Authors' Contact Information: Kewei Sha (corresponding author), University of North Texas, Denton, TX, USA; e-mail: kewei.sha@unt.edu; Kwok-Bun Yue, University of Houston-Clear Lake, Houston, TX, USA; e-mail: yue@uhcl.edu; Wei Wei, University of Houston-Downtown, Houston, TX, USA; e-mail: weiw@uhd.edu; Yalong Wu, University of Houston-Clear Lake, Houston, TX, USA; e-mail: wuy@uhcl.edu; Madhuri Koduru, University of Houston-Clear Lake, Houston, TX, USA; e-mail: KoduruM7307@uhcl.edu; Preethi Vuchuru, University of Houston-Clear Lake, Houston, TX, USA; e-mail: VuchuruP6722@uhcl.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2769-6480/2025/10-ART28

<https://doi.org/10.1145/3699965>

common goal regardless of the form it may take. In a consortium, members reach a certain level of agreement on collaboration based on members' common interests. Individually, each consortium member will have its specific interest.

Researchers have developed many collaborative platforms for various application domains. For example, **building information model (BIM)** technology is developed as a digital foundation for collaborative design for architecture and construction society [4]. **Model-based systems engineering (MBSE)** is proposed to facilitate the development of complex systems [6]. **Electronic health record systems (EHRS)** are constructed to enable information sharing and collaboration among multiple stakeholders of medical applications [16]. Coming together with the power of collaboration, there are security and privacy concerns, including secure membership management, secure data storage, authentication and authorization, privacy preservation, and auditability and trustability. Many efforts are made to secure the above collaborative platforms [4, 17, 31, 49]. For example, a cloud-based security mechanism is proposed to protect BIM collaboration systems [4]. A **Unified Modeling Language (UML)**-based approach is designed to provide security support for MBSE-based system development [31]. These solutions, however, only partially solve basic security concerns and are customized for specific application domains. They are mostly centrally governed and not sufficient to provide a secure and privacy-preserving environment for a consortium. Lately, the emergence of blockchain technology has opened a new venue to secure the consortium as shown in the recent work [1, 16, 17, 27, 49]. The nice features of blockchain such as immutability, auditability, and automation make it a suitable technology for building security solutions for consortium applications. Yet the existing solutions either focus on one specific perspective of security like authentication, or are customized to one specific application. Moreover, based on our study, none of them considers integration with the existing information systems while designing their security solutions [1, 16, 17, 27, 49].

This research aims to design a general-purpose distributed security framework, ConsortiumSec, which builds upon emerging blockchain platforms to secure consortium applications. Based on the analysis of the security needs of the consortium and each organization, the proposed framework employs a two-layer architecture, including the consortium layer and the organization layer, to systematically address the security problems for consortium-based applications and strengthen trust among consortium participants. At the organization layer, each organization makes use of its existing computing facilities to authenticate organization users, and authorize them to perform consortium operations, while storing sensitive data only within the organization itself. At the consortium layer, the consortium defines roles and access policies for organizations, manages the defined policies, stores consortium data, and enforces the security of all these steps. The two-layer architecture not only integrates the organization's existing security management into the consortium security design but also allows each organization to store its private data locally. It does not require the organization to trust other participants in the consortium fully but achieves a flexible secure data-sharing mechanism among multiple organizations with mutually approved policies.

The contribution of the article is three-fold. First, we propose a new blockchain-based distributed security framework for consortium-based applications. The proposed framework is general-purpose and provides systematic security and privacy protection to the entire consortium. It designs a novel access control mechanism as well as an efficient and secure blockchain offchain storage mechanism. Second, we implement the proposed framework based on a popular permissioned blockchain platform, Hyperledger Fabric [3]. Third, we demonstrate how to apply the proposed framework in a typical real-world consortium application.

The rest of the article is organized as follows. Section 2 presents the basic concept of consortium and security concerns in consortium-based applications. Section 3 briefs the different types of blockchain technology and the choice of consortium blockchain. Then we describe the detailed design of ConsortiumSec, the proposed distributed security framework in Section 4, followed by the illustration of a Hyperledger Fabric-based implementation of ConsortiumSec in Section 5. We showcase applying ConsortiumSec to secure a consortium MBSE project in Section 6. Finally, we conclude the article and discuss future work in Section 7.

2 Background, Motivation, and Related Work

Collaboration is pervasive in today's society. Complex projects require experts from various fields to contribute their intelligence and skills. For example, NASA works together with many research institutions, external contractors, and international collaborators to explore space [45, 46]. Multiple departments from hospitals coordinate to design treatment plans for oncology patients. A consortium is a typical form to work on collaborative projects and achieve common goals.

We summarize the characteristics of a consortium as follows. First, a consortium can be project-based and can have variant sizes. It can be formed and evolves dynamically as needed. One organization may collaborate with many different organizations in different consortia. Two organizations that collaborate in the current consortium project can become competitors in other projects. In fact, there may be competition between organizations within the same consortium project. Second, in a consortium, achieving a common goal requires organizations to maximally share resources based on a certain level of mutual trust, while each organization will also have its own interests and may not choose to fully trust others, especially when there are significant commercial interests [20]. There could be two types of lack of trust: a complete lack of trust between participants that are assumed and addressed by blockchains such as Bitcoin [33] and Ethereum [14], and a semi-trust relationship, where there is some trust because of being members of the same consortium with a common goal. Lacking trust among participating organizations can be an obstacle to a successful collaboration. Third, consortium management is important to a successful collaboration. With many participating organizations in the consortium working on a complex project, it is challenging to manage a consortium. Consortium management includes many perspectives, which include establishing a flexible platform to manage the consortium, reaching and maintaining a consensus on consortium policies, allowing each organization to have control over its own sensitive data, etc. [42]. Lack of trust further complicates consortium management.

Based on the above analysis, in addition to building trust from the collaboration history and legal agreement, it is critical to establish a secure and flexible environment that allows collaborating organizations to control their own sensitive private data. The secure and flexible environment ensures that only authorized collaborators can get access to the shared resources, and organizations having control over what to share significantly reduce the concerns of private information leakage. Moreover, the ability to audit the data access history and trace down inappropriate data access will further enhance the mutual trust among organizations [19]. Next, we analyze the security needs from both consortium and organization perspectives.

From the perspective of the consortium, only member organizations of the consortium can be allowed to perform defined operations such as **create, read, update, and delete (CRUD)**. Therefore, the consortium should *securely manage the membership* of participating organizations. The request from any member organization will be processed only after *authentication and authorization*. The authorization needs to be enforced by *access control policies*, which should be defined and agreed upon by all stakeholder organizations that will be impacted by the specific access control policy. In other words, unlike traditional access control policies for centralized organizations, consortium access control policies need to be more consensus-based, nuanced, finer-grained, distributed, decentralized, and organization-respected. For example, the consortium access control policies need consensus not only from data owners as in the recent research [12, 26] but from all stakeholder organizations. The access control mechanism needs to consider not only data-related, CRUD-type access but also operation-related access such as execution of **smart contracts (SC)**, changing consortium membership, and updating consensus-type policies. In addition, the consortium needs to *preserve the confidentiality and integrity* of the organizational data stored in the consortium-managed storage space. Whenever a violation of access happens, the consortium could be able to *detect the intrusion to the consortium applications by auditing the log and trace it down to locate the intruder* with the help of participating organizations. In order to achieve this goal, there needs security policy interoperability between the consortium and the organizations. Finally, traditional consortia are mostly centralized in transaction and data management. Having a proper degree of *decentralization*, as enabled by blockchain, vastly increases the scope and effectiveness of consortia.

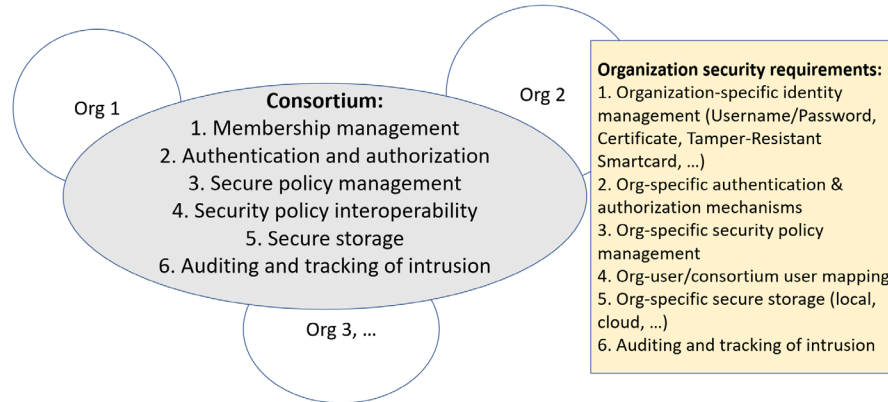


Fig. 1. Consortium security requirements analysis.

From the perspective of the organization, each organization has its own administration to manage the employees of its organization, including *employee registration, authentication, and authorization, organization policy creation and maintenance, and secure local data storage*. The organization also needs to address two *privacy concerns*. The primary concern is to *protect its intellectual property (IP) and technical secrets*, which may be the most valuable property of an organization. This requires the organization to have control of what and how much to share to which collaborating organizations in the consortium, and in what way. Moreover, it needs to decide and control the assignment of employees to the consortium project and *protect human resource related information* in some scenarios. Finally, when it is needed, the organization should be able to help the consortium in *identifying the intruder* and vice versa.

Figure 1 summarizes the security needs for consortium-based applications based on the above analysis.

Many security architectures have been developed to secure consortia [4, 8, 17, 30, 31, 49]. Examples include the cloud-based security mechanism for BIM and medical collaboration system [4], the UML-based approach for MBSE system security [31], the Software-Defined Network and Service Function Chaining mechanism for collaborative large-scale attacks mitigation [32]. These solutions, however, only partially solve basic security concerns and are customized for specific application domains. For example, these mechanisms do not build trust among collaborators and are usually centrally administrated. Customized for specific applications, these designs usually only focus on application-specific security goals and are difficult to generalize. Recently, the emerging blockchain technology enables innovation to secure the consortium as shown in the recent work [1, 16, 17, 19, 27, 35, 49]. The nice features of blockchain such as immutability, auditability, and automation make it a suitable technology for building security solutions for consortium applications. However, the existing blockchain-based security solutions either focus on one specific perspective of security (e.g., authentication [27], authorization [1, 19], etc.) or are customized to one specific application (e.g., Internet of Things [1], BIM [17, 49], EHRS [16], etc.). Even though many of them claim themselves as general-purpose security frameworks, they barely provide comprehensive security solutions. Moreover, none of them consider integrating the organization's existing information systems while designing their security solutions. This makes them difficult to deploy in real-world applications.

Strengthening trust among collaborating parties is crucial. To address this, behavior trust is introduced as a method to evaluate a device's trustworthiness based on its behavior within a specific context at a given time [5]. Various trust models have been proposed to assess, evaluate, and manage trust, relying on QoS and social trust attributes [5, 11, 34, 37, 44]. A blockchain-based trust model has also been proposed to generate precise trust information for virtual communities, making it difficult for untrustworthy actors to manipulate the trust

score [11]. Wang and Wu propose a general and extensible blockchain-based multidimensional trust management model [44]. This model integrates the multicriteria evaluation data to measure the dynamic trust performance and filters out malicious or unprofessional evaluations to ensure the credibility of trust management. It leverages blockchain technology and SC to store trust evidence, share trust information, and execute multisource trust fusion automatically, addressing issues like information opacity and the single point of failure of the traditional centralized trust model. In this article, while we mainly address security-related issues, we examine trust from two perspectives. First, in consortium settings where multiple organizations collaborate, we assume a semi-trust relationship among collaborating organizations. This assumption holds because collaborators generally possess a certain level of mutual trust. However, we do not evaluate trust using a behavior trust model. Second, the suitability based on immutability and the consensus algorithm inherent in blockchain technology further reinforce trust among collaborating organizations from the perspective of behavior trust. Security is enforced through authentication and authorization, which pertains to identity trust.

This work aims to design a distributed security framework that is general-purpose but can be easily customized for various consortium-based applications. The main goal of the proposed framework is to provide a secure collaboration platform, and trust will be enhanced through the immutability, auditability, and consensus mechanisms inherent in blockchain technology.

3 Blockchain for Consortium Security

Blockchain is a decentralized and distributed ledger that securely records digital transactions. As a disruptive technology, blockchain technology brings the evolution of cryptocurrency and has extended its application to many fields. In its first application as Bitcoin [33], the design of blockchain focused on sending, receiving, and trading cryptocurrency. Since then blockchain has evolved continuously. Being the representative of Blockchain 2.0, Ethereum advanced the blockchain technology significantly by introducing SC [14]. This opens the door for blockchain applications in many other domains beyond cryptocurrency, especially in digital economy [21]. Using SC, Ethereum also aims to build a digital ecosystem that can be used as a basis for other decentralized projects. The new generation blockchain targets to solve scalability, interoperability, and performance issues [43]. These blockchains can also be classified into two categories: permissioned and permissionless. Permissionless blockchains like Bitcoin allow everyone to join the blockchain and are entirely decentralized. They are also called public blockchains. Permissioned blockchains implement a mechanism that determines which organizations and nodes can join the blockchain. There are two types of permissioned blockchains, private blockchain which is controlled by one organization, and consortium blockchain which is controlled by a group of organizations in the consortium. Permissionless blockchains tend to have longer validation times for new data and thus suffer with scalability (low number of transactions per second), and permissioned blockchains are more vulnerable to fraud and bad actors than the public ones not only because they have a smaller number of actors and nodes but also some of them adopt partially decentralized consensus algorithms that include only a smaller selective set of nodes in the consensus process [28].

The popularity of blockchain technologies comes with its unique features. Blockchain is a decentralized and distributed technology. The distributed consensus algorithms avoid trusting a centralized authority. Blockchain records all transactions in history. The distributed ledger, especially the permissionless blockchain, achieves transparency and immutability because of distributed storage and consensus algorithms. The complete, rich, and immutable transaction history also makes auditability possible. Blockchain embeds security mechanisms such as storing encrypted data and distributed consensus. It has a good level of both confidentiality and integrity. Moreover, the SC enables automated transaction management. Additional security protection can be incorporated into blockchain technologies when needed. With the above features, researchers start to investigate using blockchain to secure consortium applications [1, 16, 17, 19, 27, 35, 49]. These efforts advance the frontier of blockchain-based security solutions, while none of them systematically study a general-purpose framework for consortium security.

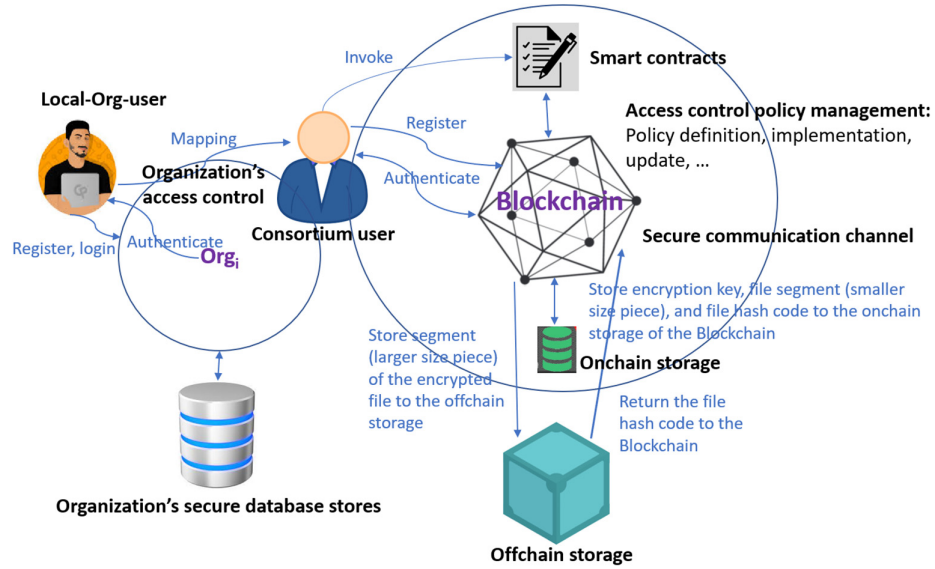


Fig. 2. The design of ConsortiumSec.

Moreover, even though blockchain is recognized as a foundation technology for building consortium applications, proper security assurance remains a main challenge for building consortium blockchain [15]

The designs of blockchains in different generations are customized for various targeted applications although they share most of the above-discussed features. Therefore, when we propose a general-purpose distributed security framework based on blockchain for consortium applications, we need to select an appropriate blockchain as the preferred platform to implement the proposed framework. Generally speaking, we prefer a blockchain that is permissioned (consortium blockchain fits consortium applications because consortium applications are usually not open to the public), automated (it supports SC), and security and privacy enhanced (it has enhanced security features and privacy-preserving mechanisms).

4 Design of ConsortiumSec: A Blockchain-Based Distributed Security Framework

In this article, we propose a general-purpose blockchain-based distributed security framework, ConsortiumSec to provide a systematic secure solution for consortium applications. As depicted in Figure 2, the framework consists of two layers, the *organization layer* and the *consortium layer* to satisfy both the security and privacy needs of organizations and the consortium. The organization layer builds upon the existing computing facility of the organization, while the consortium layer is managed based on a consortium blockchain. At each layer, we aim to realize security goals identified in Section 2 that include secure membership and policy management, authentication and authorization, secure storage and communication, privacy protection, and auditing and tracing of attacks. Next, we detail how these security goals are achieved in ConsortiumSec.

4.1 Membership and Security Policy Management

In Figure 2, at the consortium layer, each organization acts as a consortium member. There are many ways to manage consortium membership and access policies. First, the consortium can have a centralized organization that takes care of membership management and consortium policies. This design can be a suitable choice when the centralized organization has authority that all organizations trust to manage the consortium. It risks when the centralized organization is malicious or incompetent but it simplifies consortium management. This design

may not be preferred as most consortium applications are naturally managed in a distributed format. Second, the consortium may have a central authority that can select a set of organizations to collaboratively manage the consortium membership and policies. This is suitable for consortia with hierarchical-organizational structures. With specifically designed consensus mechanisms, the consortium registers consortium members in two ways. In one way, the consortium **certificate authority (CA)**, which could be a CA trusted by consortium members, issues a certificate and an identification item (e.g., a private key) for each member. On the other way, the consortium CA validates the certificate issued by the organization CA to register the member. Currently, this design may not be supported directly by many permission-based blockchains including Hyperledger Fabric.

Third, all organizations can work together to decide the consortium management policies by securely agreeing upon the membership management, access policies, etc. Application-layer secure distributed voting algorithms can be applied to reach a consensus. The lower-layer consensus algorithms of the blockchain can naturally support the implementation of the application-layer algorithm. This choice is suitable for consortia with flat organizational structures, where organizations have no hierarchy and have similar importance in making consortium decisions, and it signifies limited or close to no mutual trust among organizations. It could achieve transparent and totally decentralized management among participating organizations.

Each organization could have multiple organization's local users participating in the consortium collaboration. After the organization joins the consortium, the organization's local users will perform the organization's consortium tasks on behalf of the organization with the permission of the organization. When organization local users perform consortium tasks, they act in the role of **consortium user (CU)**. In some applications, one organization's local user may play multiple CU roles, and one CU role may be performed by multiple local users of the same organization. There will be organization-specific mechanisms to assign organization local users to be CU. Each organization can use its existing approach to managing its local users' identities at the organization layer. These approaches include username/password, certificate, temper-resistant smart card, tokens, biometrics, etc.

4.2 User Authentication and Authorization

Authentication and identity are closely related. At the consortium layer, a common approach to authenticate the consortium members is to use certificate-based authentication using the **Public Key Infrastructure (PKI)**. At the organization layer, each organization picks up a corresponding authentication mechanism based on its specific identity management approach. It can be either PKI or other mechanisms. For example, when a username/password is used as the user credential, a multi-factor authentication protocol can be used. When biometrics is used as a user's credential, biometrics authentication protocols will be adopted.

Authorization is critical to control the access consortium asset and perform consortium operations. The consortium can choose appropriate access control mechanisms [48] to authorize CU based on the needs of applications. Considering the complexity of consortium applications, we suggest **attribute-based access control (ABAC)** [22] because it can provide flexible, fine-grained, precise access control. Some details of our blockchain-based distributed ABAC are presented in [46]. With well-designed ABAC policies, the consortium can control CU interacting with the blockchain by invoking specific SC that are designed to support application-specific functions, such as creating, reading, updating, and deleting the consortium data, installing new SC, managing CU, etc. At each organization, the organization's existing local access control will be utilized to grant its **local organization users (LOU)** to access the consortium application. Only after the access is granted, the LOU will be mapped to a CU and plays the specific CU role on behalf of the organization.

A LOU needs to go through a series of authentications and authorizations deployed at both the organization layer and the consortium layer before it can perform consortium tasks. Algorithm 1 summarizes the process that allows a LOU to execute blockchain transactions with SC invocation through a client application. The algorithm also records user activities for future auditing and tracking of attacks. For brevity, pseudo-code for handling

Algorithm 1: Executing a Client Application (C) with Smart Contract Invocations by a Local Organization User (LOU) and its Credential (LOU Credential) on a Consortium Blockchain

```

AUTHN_AUTHZ( $LOU, LOU\_Credential$ )
[1]  $Org_i$  authenticate  $LOU$  based on  $LOU\_Credential$ 
[2]  $CU \leftarrow Org_i$  fetches consortium user role of  $LOU$ 
[3]  $Org_i$  fetches  $CU$ 's credential of  $LOU$  for application  $C$ 
[4]  $CU$  sends an authentication request to the blockchain
[5] Blockchain authenticates  $CU$ 's credential
for every smart contract  $SC_s$  do
  [6]  $CU$  prepares and submits transactions by invoking  $SC_s$ 
  [7]  $Org_i$  records event of invoking  $SC_s$  with pending execution status in local secure storage
  [8] Blockchain checks access policies for  $SC_s$ 
  if (access is granted by blockchain) then
    [9]  $SC_s$  executes
    [10] Blockchain records the transaction of  $SC_s$ 
    [11]  $SC_s$  returns the results and execution status to  $C$ 
  end if
  [12]  $Org_i$  stores the relevant result of  $SC_s$  in the local secure storage
  [13]  $Org_i$  updates  $SC_s$  status in local secure storage
end for
[14] Use the result returned by  $SC_s$ 

```

insufficient credentials and roles is not included in the algorithm. If permission is not granted in any step, the client application C fails. House cleaning in the local organization is also not included.

In the algorithm, the organization (Org_i) first authenticates the LOU based on the LOU's credential (step 1). If successful, LOU is recognized. Otherwise, the client application suspends. Then, Org_i fetches the CU role and credential of the LOU for the application C from its local storage. The LOU is now mapped to a CU, CU (steps [2] and [3]). As a CU, CU sends an authentication request to the blockchain in step [4]. The blockchain validates and confirms CU 's credential in step [5]. This allows CU to submit transactions by invoking SC. Whenever the CU maintains a valid authenticated status and has a need of performing blockchain transactions, it prepares and submits a smart contract SC_s (step [6]) to the blockchain. Org_i records the event of the SC invocation (step [7]) with pending execution status. The blockchain checks the access control policy for the submitted SC. The smart contract executes if access is granted (step [9]). In step [10] the blockchain records the transaction of SC_s , which can be done by defining common auditing helping methods in the chaincode. Then it returns the execution results and status back to the client application in step [11]. Next, Org_i stores the relevant result of the SC in the local secure storage (step [12]) and updates the status of SC_s (step [13]). The record will be used in auditing and tracking attacks as described in Section 4.5. Finally, the client application proceeds by using the result returned by SC_s (Step 14). In addition, organization Org_i stores CU credential, OU and CU mapping, and logs in its local secure storage, which is detailed in Section 4.3.

4.3 Secure Storage and Communications

All data storage and communications are secured in consortiumSec. When the data is stored locally in a specific organization, the organization can utilize its existing secure storage mechanisms like a secure database to manage sensitive data that include but are not limited to the organization's private data, any organization's data related

Algorithm 2: Algorithms to Securely Store a File to Offchain Storage

```

STOREFILETOOFFCHAINSTORAGE(assetKey, F)
[1]  $key \leftarrow$  a randomly generated key
[2]  $F_e \leftarrow \text{encrypt}(F, key)$ 
[3]  $head \leftarrow F_e[0..HEAD\_SIZE - 1]$ 
[4]  $tail \leftarrow F_e[HEAD\_SIZE, \text{size}(F_e) - 1]$ 
[5] store tail into IPFS and get IPFS hash address
[6] create a blockchain asset  $A$  that includes head, key,
    IPFS hash address ( $cid$ ), etc.
[7] invoke a smart contract,  $sc.PutState(assetKey, A)$ ,
    to store the asset in the blockchain
[8] return true, if no exception

```

to the consortium application, LOU information, LOU and CU mapping, CU credential, all certificates needed for security, etc.

Blockchains provide onchain storage to store consortium application data. All data stored in the blockchain's onchain storage are in encrypted format for confidentiality in consortiumSec. Due to the high cost of storing and accessing large-size data in the blockchain, researchers propose to use offchain storage to store large-size files [29]. Especially, **Interplanetary File System (IPFS)** is widely used as blockchain offchain storage [23]. When the data is stored offchain in IPFS, however, there raise concerns about bypassing the blockchain's access control policies because of two causes, i.e., the public IPFS network is easily accessible as a decentralized and shared storage, and it does not provide the same level of security as blockchain. Even a private IPFS network is protected by a sole swarm key that is not likely to change and can thus provide only a low level of access protection. For example, attackers may gain the IPFS hash address to access the data stored offchain. To address this issue, we encrypt the data using a one-time key when it is stored in offchain IPFS and then store the key and the IPFS hash address in the blockchain's onchain storage. However, the encryption of IPFS content can be broken by future technology. We need a more sophisticated algorithm to enhance the security of the offchain storage. In our design, we further improve the security of offchain storage by designing a new mechanism as described in Algorithm 2. It targets to ensure that getting access to data stored in IPFS will not be able to retrieve the data stored through the blockchain. We accomplish it by distributing the data storage in both IPFS and blockchain.

The secure offchain storage algorithm takes the file as the input. The term file represents the data to be stored in the blockchain. It could be in the format of a file or a blockchain asset. It consists of several steps to securely store the file in blockchain with offchain storage, including (1) generating a random one-time key and encrypting the file using the generated key (lines [1] and [2]); (2) splitting the file into two parts (head and tail) based on the known value of head size, specified as *HEAD_SIZE*, which is usually a small amount like 16 Bytes (lines [3] and [4]); (3) storing tail in IPFS (line [5]); (4) storing head, one-time key, and tail's IPFS hash address as a blockchain asset in the onchain storage (lines [6] and [7]); (5) returning a Boolean value indicating if the operations are successful (line [8]).

Particularly, in our design, the head size is constant, defined by *HEAD_SIZE*, which is typically a small number such as 16 bytes. Storing a piece of F_e in the blockchain ensures that even though attackers may get the tail part of F_e , it will not have the entire F_e for proper decryption. Note that the one-asset encryption key is also stored in the blockchain for even stronger protection. If there is a need for a larger head, the algorithms can be simply changed to store a 64-byte head, which still uses small storage space. Furthermore, the algorithms can be adapted to handle varying head sizes by simply storing the head size as a meta-data of the asset recorded in the blockchain.

Algorithm 3: Algorithms to Retrieve a File from the Offchain Storage

```

RETRIEVEFILEFROMOFFCHAINSTORAGE(assetKey)
[1] asset  $A \leftarrow sc.GetState(assetKey)$ 
if ( $A \neq nil$ ) then
    [2] tail  $\leftarrow IPFS.Get(A.cid)$ 
    [3] head  $\leftarrow A.head$ 
    [4]  $F_e \leftarrow head + tail$ 
    [5] key  $\leftarrow A.key$ 
    [6]  $F \leftarrow Decrypt(F_e, key)$ 
    [7] return  $F$ 
else
    [8] return  $nil$ 
end if

```

When the user needs to retrieve a file stored in the blockchain using Algorithm 2, Algorithm 3 is designed to achieve the goal. The user first needs to get authenticated and authorized as described in Section 4.2, then it invokes a smart contract $sc.GetState(assetKey)$, where the $assetKey$ associated with the file is passed as a parameter (line [1]). If the $assetKey$ is valid, the SC will return the corresponding asset information stored in the blockchain onchain storage that includes the encrypted file head (head), the key used to encrypt the file (key), and the file tail's hash address in the offchain storage (cid). With cid , we are able to retrieve the tail of the file from the offchain storage such as IPFS (line [2]). Combining the head and tail, we get the entire encrypted file (F_e) (lines [3] and [4]). Then we can use the key to decrypt (F_e) and obtain and return the original file F stored in the blockchain (lines [5] and [7]).

When the size of sharing information increases, the information stored in the on-chain storage remains the same size, consisting of the head and the hash code. However, the size stored in the off-chain storage will increase with the increased information size. We believe off-chain storage can support large-size file storage, therefore, enabling the system to maintain good scalability as the size of sharing information increases. In general, the scalability of our framework is at the same level as Hyperledger Fabric, as it does not introduce any additional bottlenecks beyond the intrinsic scalability consideration of Fabric.

Our pilot implementation does not involve experimenting with a very large volume of assets and collaborators, so there is no experimental data on scalability. However, ConsortiumSec just uses typical SC along with IPFS. Both Hyperledger Fabric and IPFS are well-known for their high scalability. No component in ConsortiumSec introduces any additional bottlenecks beyond the intrinsic scalability considerations of Fabric and IPFS. Consequently, we do not expect scalability issues in ConsortiumSec to differ from those in typical Fabric applications.

Besides secure storage, secure communications can be achieved based on technologies that can build secure channels such as Secure Sockets Layer and **Transport Layer Security (TLS)**. Many types of blockchains embed these security protocols in their implementations.

4.4 Privacy Protection

ConsortiumSec achieves data privacy protection by allowing organizations to control the storage and sharing of their private data. First, the sensitive data is only stored locally in the organization's secure storage. For example, organizations may not share their detailed designs and implementation information that has valuable IP even with their collaborating organizations in the consortium. Instead, they can choose to store these data locally only. For consortium collaboration, the organization can generate an anonymized and IP-sanitized version such as an abstract of the design for the consortium. Second, when the data is stored in the blockchain, the organization

can specify an access control policy that will be enforced to limit access to the stored data. Per our previous discussion in Section 4.2, fine-grained access control policies like ABAC policies can be formulated based on the collaboration among organizations. In addition, some consortium blockchains like Hyperledger Fabric provide privacy-preserving mechanisms such as separated and isolated channels and **private data collections (PDC)** [9]. These mechanisms provide extra protection that enables organizations to manage with whom the data will be shared.

The organization's employee (LOU) identity will be hidden from collaborating organizations. ConsortiumSec protects identity privacy by two means. First, each organization manages a flexible and dynamic mapping between the organization's employee identity and the CU. For a certain phase during the consortium collaboration, there could be cases in that multiple employees are mapped to a single CU and one organization employee may play multiple CU roles for specific applications. Collaborating organizations only view the CU who performs the collaborative tasks and cannot know who the actual employee is. Moreover, some consortium blockchains provide innovative mechanisms such as mixed identity and zero knowledge proof [39] to preserve identity privacy.

4.5 Auditing and Tracing of Attacks

Blockchains are appending systems. Data is organized in an ever-growing list of blocks with each block referencing the previous one. After data is stored in the blockchains, it will not be eliminated or modified. Newly added data will be appended to the blockchain but not to replace the old data. The immutable blockchain systems can provide a single source of truth [45]. This makes blockchain a suitable technology for auditing and tracking attacks as the complete history of the data provides excellent attack analysis values.

In addition, whenever necessary, consortium applications can record extra information that helps to investigate attacks (e.g., authentication and authorization activities) and store them in the blockchain and the organization's secure storage. Many blockchain platforms automatically record a treasure for transaction related data useful for detecting, tracing, and auditing attacks. For example, based on Hyperledger Fabric, applications like Block Reader [38], allow easy extractions of relevant transaction data and metadata from a block [50]. This includes data on the transactions (e.g., name, type, channel), and their associated SC's executions (name, functions invoked, function parameters, read sets, write sets, invoker's information, endorsement information, etc.) [50]. In local organizations, security audit data can also be designed to collaborate with blockchain audit data. For example, a local organization may store the LOU that invokes a transaction, the peer node the transaction is submitted to, the IP address used by LOU, etc. A carefully designed set of collaborative audit data collected by the blockchain and the local organization can provide an effective basis for detecting, analyzing, and remediating attacks.

When an investigation is issued, investigators can audit these stored records to trace, identify, and locate intruders, i.e., we can link audit traces in the blockchain and the organization's security audit data to trace down the attacks.

4.6 Discussion and Open Research Issues

The proposed blockchain-based distributed security framework is a general framework, which can be applied to many consortium applications. In this two-layer security management design, we allow individual organization and consortium to manage their access control independently and organizations can keep the flexibility of integrating their existing security administration schemes.

In the current architecture, all collaborators must be part of the Hyperledger. This setup meets the requirements of consortium applications, i.e., they are not open to the general public but require memberships. For example, in one of the real-world applications, we target a NASA engineering application only involving collaborations among specific companies and organizations. However, connecting the consortium blockchain to other consortiums or public blockchains could be the next logical extension of ConsortiumSec, which we plan to explore in the future.

Additionally, the current framework requires developers with expertise both in security and blockchain. Our future plans include developing and standardizing the proposed framework to provide a general platform that is open to and can support a broader range of consortium collaboration applications. This will be achieved by incorporating user-friendly interfaces. We will also further explore optimizations that can support applications with requirements on scalability, heterogeneity, and data consistency.

Moreover, we may consider designing cross-layer distributed access control mechanisms that jointly consider the security needs and optimize the security designs for both organization and consortium layers, especially for newly created consortium applications that do not need to deal with the legacy IT infrastructures at each organization. Besides, we recognize that behavior trust may significantly impact the success of consortium collaboration. In the future, we will explore ways to enhance our design to further strengthen behavior trust.

When the access control policies are designed, fine-grained access control policies are expected for complex consortium applications, while they also result in design complexity and impact the performance of the entire system. There requires a study of the tradeoff between the granularity and the overhead of access control policies.

ConsortiumSec also paves the road to investigate consortium governance issues, which include consortium membership management, consortium structure, consortium security and privacy, consortium operational governance, etc. Because of the complexity of consortium governance, we need to reexamine and expand the proposed framework from the perspective of consortium governance. On the other hand, the unique mechanism in blockchain like SC provides us with a new perspective to design and examine novel consortium governance.

Analyzing blockchain data for security audition [40], attack tracking [18], and forensic investigation [25] has caught researcher's attention recently. For example, systems like BlockSci [24] have been proposed as blockchain analysis platforms. However, blockchain data analytics is still in its early stages because of the high cost and many privacy concerns. Moreover, blockchains are considered relatively slow systems and have more data integrity issues compared with traditional database systems [46].

5 Fabric-Based Implementation of ConsortiumSec

ConsortiumSec is a general-purpose blockchain-based secure framework that aims to systematically solve the security issues in consortium applications, and it requires expertise in both blockchain technology and security principles to implement the proposed framework.

ConsortiumSec can be implemented on various blockchain platforms, but a permissioned consortium blockchain is the preferred option. This is because ConsortiumSec is specifically designed for consortium applications, and consortium blockchains are inherently suited to meet the requirements of such applications. Popular permissioned blockchain platforms include Quorum [7], Corda [10], Hyperledger Fabric, Hyperledger Sawtooth [2], etc.

We choose to implement ConsortiumSec based on Hyperledger Fabric (Fabric) because of the following considerations. First, Fabric is the most popular permissioned blockchain where data stored in Fabric is not open to the public, which matches the needs of Consortium applications. Second, Fabric supports a native coarse-grained access control mechanism, which can be used as a building block for refined security and privacy assurance. It provides rich embedded features for security and privacy protection, including separated channel policies, CA, **Membership Service Providers (MSP)**, PDC, TLS-based secure communication, etc. Third, Fabric supports several programming languages, including GoLang [47], Java, and JavaScript, to implement smart contracts. Besides, there is great community support and sample smart contracts. Fourth, it enables mechanisms like offchain storage needed by consortium applications to store large-size application data. Last but not the least, Fabric works with tools like IBM Blockchain Extension to expedite the development of blockchain applications. In summary, the above characteristics of Fabric not only facilitate ConsortiumSec prototyping but also empower the design and implementation of a blockchain-based distributed ABAC. Permissioned blockchains like Hyperledger Fabric consume relatively fewer resources due to their permission-based nature and the choices of consensus algorithms, our proposed framework does not introduce any additional bottlenecks beyond those inherent to Hyperledger

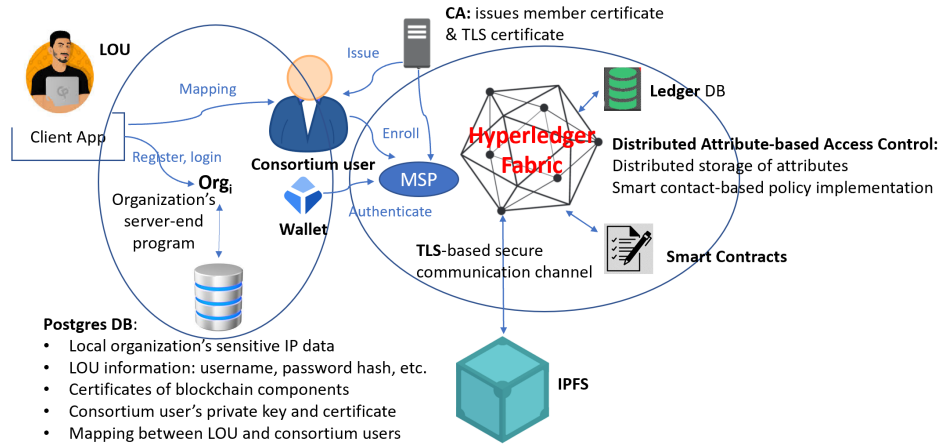


Fig. 3. Hyperledger Fabric-based implementation of ConsortiumSec.

Fabric. Although blockchain is generally resource-intensive, it has been proven to be scalable. Additionally, the consortium applications we target involve collaboration among organizations that are likely to be reasonably large-scale but not extremely large-scale. With these considerations in mind, we believe our proposed framework will be sufficiently scalable to support consortium applications.

Figure 3 depicts the Hyperledger Fabric-based implementation of the proposed ConsortiumSec, which consists of a blockchain application for each organization and a Fabric network. The blockchain application includes an organizational client app and an organization's server-end program. The client app provides an interface for LOU to perform tasks of consortium projects while the server-end program implements the business logic that is needed by the organization to interact with the blockchain. Below we demonstrate the implementation of several key security controls.

Blockchain Settings. A Hyperledger Fabric instance is set up which includes orderer nodes, peer nodes, CAs, MSPs, channels, etc. We configure the CouchDB as the World State database in Fabric. An IPFS is created and connected to the Fabric instance as the offchain storage. TLS will be enabled based on the TLS certificate from the TLS CA to provide secure communication among all Fabric nodes. To simplify the implementation, we use only one channel(ledger) and configure three organizations to demonstrate our ideas.

User Registration. LOUs are registered by the local organization administrator who administers the organization's computing infrastructure, including the secure database, networking, and security. To simplify the implementation, we adopt password-based identity management. When the local organization administrator registers an LOU, a pair of username and password is created and shared with the LOU. Meanwhile, the username and password hash is stored in the organization's local secure database. Here we set up a Postgres database [13] as the organization's local secure database.

Similarly, CU are registered and enrolled by the administrators of Fabric organizations. In this process, we use the Fabric CA to generate a private key and certificate pair when registering a CU who will perform consortium tasks on behalf of a local organization. The private key will be sent to the local organization that the created CU belongs to and it will be stored in the local organization's Postgres database together with the certificate. The certificate will also be distributed to multiple blockchain MSPs when enrolling the created CU into the blockchain.

User Authentication. LOUs log in through an interface provided by the client app for its organizations. An organizational server-end program authenticates the user by verifying its identity. In this implementation, we adopt a username/password-based two-factor authentication with the help of the Postgres database. After receiving LOU's authentication request, the server-end program compares the received password hash with the

password hash stored in the Postgres database. Following successful authentication, the server-end program fetches the mapping between the organization user and the CU from the Postgres database. Next, the server-end program builds a dynamic in-memory Wallet using the stored credentials fetched from the Postgres database including the CU's private key and the certificate. An authentication request including the CU's digital signature and its certificate will be sent to the Fabric MSP. Fabric's MSP authenticates the CU by validating its certificate and digital signature.

User Authorization. The user authorization is achieved in three steps. First, the organization's server-end program manages the mapping between the organization user and the CU. It authorizes an organizational user to act as a CU. Second, MSP authenticates the CU, and this authorizes the user to invoke appropriate smart contracts. Third, before the SC performs its operations (e.g., read, write, create, update, etc.) to interact with the blockchain, it will check application-specific ABAC policies. In this study, ABAC policy checking is also implemented using smart contract. The checking results grant permission for CU's operation. Please find the detailed design and implementation of distributed ABAC based on Fabric in [46].

Secure Storage. ConsortiumSec encrypts all data stored in the onchain storage. We implement Algorithms 2 and 3 as described in Section 4.3 to secure any file stored in IPFS. As described above, we utilize the Postgres database to simplify the implementation of ConsortiumSec. First, it is used to authenticate LOU. Second, it stores CU credentials that are needed to build the dynamic in-memory wallet. We believe it is more secure to store CU credentials in a secure database like Postgres than in the file system. A similar practice has been seen in many Fabric experiments. Third, it eases the local organization to authorize the LOU using the stored mapping between LOU and CU. This simply demonstrates the ideas of access control in local organizations. In a production system, if a local organization has other access control mechanisms, they can be easily integrated into the implementation to replace the simple mapping. Fourth, it stores application user activities that can be used for auditing and tracking attacks. Finally, it secures local organizations' sensitive data using appropriate access control mechanisms. Please note that the ConsortiumSec framework can work with any encryption algorithm. In the pilot implementation of Algorithms 2 and 3, we use efficient symmetric encryption, the widely used and reliable AES-256 encryption.

To summarize, ConsortiumSec leverages typical SC alongside the use of IPFS. Both Hyperledger Fabric and IPFS are well-known for their high scalability. No component in ConsortiumSec introduces any additional bottlenecks beyond the intrinsic scalability consideration of Fabric and IPFS. Therefore, we do not expect scalability issues in ConsortiumSec that would differ from typical Fabric applications. However, it is important to note that we have not conducted large-scale performance testing. In the future, we plan to extensively evaluate the proposed framework, focusing on resource usage, scalability, and delay. We intend to conduct interviews and surveys to assess the user-friendliness of the proposed framework. Our pilot experimentation indicates challenges in implementing an appropriately granular ABAC system that can effectively assure consortium rules and security, particularly on blockchain platforms. Moving forward, we aim to explore and design an optimal solution for a general-purpose ABAC package that addresses these challenges effectively.

6 Case Study: ConsortiumSec for a Change Request Process

In this section, we showcase how to apply ConsortiumSec to secure a consortium application, a blockchain-based repository to support MBSE [45]. In this pilot consortium application, multiple organizations work on a collaborative project that develops models needed to support NASA's projects. The blockchain is used as a trustful and secure repository that enables data sharing among multiple organizations, including NASA, collaborating universities, and external contractors. Selected models, project requirements, and related metadata are stored in our prototype. Figure 4 depicts a **change request (CR)** process in **Business Process Modeling Notation (BPMN)**, which is a typical scenario that needs multiple organizations' collaboration in decision-making. A CR is created when project requirements change and it requires approval based on multiple program collaborators' inputs. The **chief integration system engineer (CISE)** in one organization will create a draft CR and make it

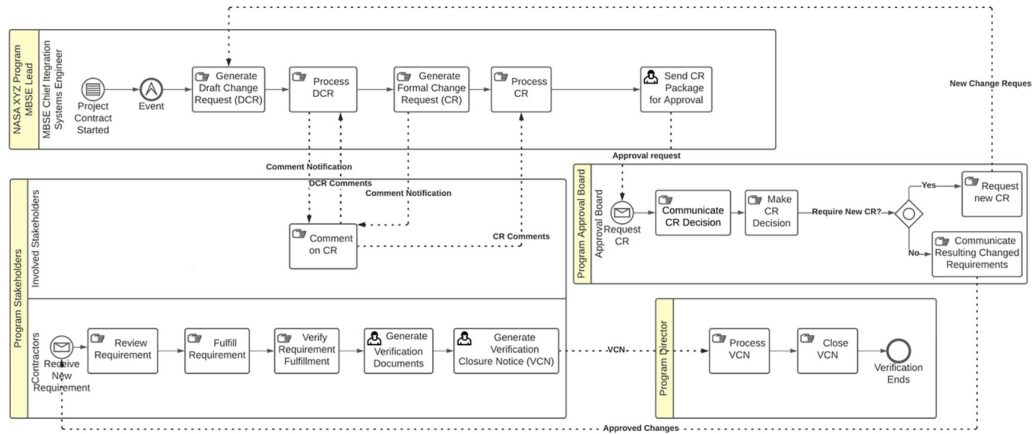


Fig. 4. The CR process for MBSE models and requirements in BPMN.

visible in the blockchain. The program stakeholders will comment on the CR. Then the CISE generates a formal CR by integrating the received comments and sends it to the program approval board for approval. The board will follow the procedure to approve the CR or request a new CR. Our goal is to demonstrate how ConsortiumSec can be applied to secure the entire process.

Building upon the Fabric-based ConsortiumSec implementation described in Section 5, we implement the CR approval process with an application-specific client app, server-end program for each participating organization, the Postgres database for application data, and SC to interact with Fabric. Moreover, the case study enforces access control by designing fine-grained ABAC policies, which mainly consist of three steps: (1) define the high-level ABAC policies for the consortium application using **abbreviated language for authorization (ALFA)** [36]; (2) implement the defined ABAC policies using SC based on the Fabric platform; (3) test and verify the implementations.

The most widely used language to model ABAC is **eXtensible Access Control Markup Language (XACML)** [41]. However, the complexity of XACML makes it difficult for application users, who will struggle to understand many technical terms. Instead, ALFA is a popular language to formally define high-level access control policies. ALFA policies provide a user-friendly interface that enables us to effectively communicate with application users who may not understand either XACML or other lower-level programming languages such as GoLang and Javascript. We capture the application access control requirements and define precise ALFA policies for them. For the entire CR process, we formulate five major access control policies. These policies include creating a CR, reading a CR, updating a CR, withdrawing a CR, and **inserting a CR decision (insertACRDecision)**. This section presents an example of one ABAC policy in ALFA-like format, insertACRDecision, and its implementation in a SC. This ABAC policy specifies the rules that will allow a CU to insert its decision on whether or not to accept the CR. In addition, while inserting the CR decision, the CU cannot change anything else about the CR. Therefore, this operation is a partial update operation, which allows changing the value of only one specific field.

Figure 5 shows the insertACRDecision policy in the ALFA-like format. This policy requires that only the CU playing the *manager* role from the control board of the participating organization can insert a valid CR decision. We define one rule with three conditions (lines 13–16 in Figure 5) in the policy. Two conditions (lines 12 and 16) check the value of ABAC attributes and are in the format of ALFA, while the other condition (lines 13–15) requires a function call to retrieve some specific attributes, which is currently not supported by ALFA. As function calls are necessary to retrieve specific attributes in consortium applications, in the future, We will work on developing ALFA extensions to support extra requirements needed by consortium applications. After the insertACRDecision

```

1 namespace mcba {
2   import mcba.attributes.*
3   policy InsertACRDecision {
4     target
5     clause action.AName == "InsertACRDecision"
6     clause action.newobject.BAT == "ChangeRequest"
7     clause action.ATId == "PU"
8
9     apply denyOverrides
10    rule canInsertACRDecision {
11      permit
12      condition
13        invoker.organization == action.newobject.project.controlboard
14        //The function getUserRole(organization o, user u) gets the
15        //role of the user u of the organization o.(cannot define in alfa)
16        //The function getUserRole(action.newobject.project.controlboard,user)=="manager"
17        && not (action.newobject.CRDDecisions == '')
18    }
19  }

```

Fig. 5. A sample ABAC policy in ALFA: insertACRDecision.

```

//Smart contract for inserting a Change Request decision by manager of control panel.
//partial: for space reason,keep only the portion of code for checking ABAC ALFA policy.
//Code for other logics, including error checking, are removed and shown as // ...
func (s *SmartContract) InsertACRDecision(ctx contractapi.TransactionContextInterface,
    assetActionValue string) (string, error) {
    assetActionU := new(AssetAction)
    err := json.Unmarshal([]byte(assetActionValue), assetActionU)
    // The ALFA action attributes are now in the variable assetActionU
    // ... error handling code.
    // Getting the ABAC attributes: action.object.project, which is stored as metadata
    // in the blockchain.
    projectMetadataKey, _ := ctx.GetStub().CreateCompositeKey("Project",
        []string{assetActionU.ProjectID})
    projectMetadataJson, err := s.getProjectMetadata(ctx, projectMetadataKey)
    // ... error handling code.
    projectMetadata := new(ProjectMetadata)
    // ... error handling code.
    // ABAC attribute: invoker.organization.
    userOrganization := s.GetIdentityAttribute(ctx, "organization")
    // ABAC attribute: invoker (i.e. user)
    user := s.GetIdentityAttribute(ctx, "hf.EnrollmentID")
    // userRole is the value return from calling getUserRole
    // (action.newobject.project.controlboard,user) in the ALFA policy.
    // The actual method implemented here is a more general version.
    userRole := s.GetUserRole(projectMetadata.Project.OrgUserRoles,user,userOrganization)
    // Check the ALFA rule canInsertACRDecision for permission with result
    // saved in the variable isAuthorized.
    isAuthorized :=
        // ALFA rule canInsertACRDecision condition [1]
        (assetActionU.AName == "InsertACRDecision" && assetActionU.BAT=="ChangeRequest"
        && assetActionU.actionId == "PU"
        // ALFA rule canInsertACRDecision condition [2]
        && s.Contains(s.GetOrgId(projectMetadata.Project.OrgRoles,"controlboard"),
        userOrganization) && userRole == "manager"
        // ALFA rule canInsertACRDecision condition [3]
        && assetActionU.BA,CRDecision != nil )
    // ...
}

```

Fig. 6. A sample of partial SC code for the insertACRDecision policy.

policy is defined in ALFA-like format, it is implemented using a SC as depicted in Figure 6. For space reasons, we keep only the portion of the code for checking the ABAC ALFA policy.

We have designed many testing cases to verify the correctness and effectiveness of the ABAC policy implementation. The testing results show that ConsortiumSec can provide comprehensive security and privacy protection to the entire CR process.

7 Conclusion and Future Scope

In this study, we proposed ConsortiumSec, a novel blockchain-based distributed security framework that provides systematic security and privacy protection for consortium applications. ConsortiumSec considers unique security and privacy requirements of different stakeholders in consortium applications, including identity management, authentication, and authorization, secure data storage, and intrusion tracking and audition. It builds upon emerging blockchain technologies that provide unique features such as a single source of truth, distributed and decentralized governance, flexible SC, immutable history, etc. We then implement and prototype the ConsortiumSec framework based on the Hyperledger Fabric platform and conduct a case study to test ConsortiumSec. The testing results show the effectiveness and flexibility of ConsortiumSec.

In the future, we plan to extend ConsortiumSec by addressing issues denoted in the following three perspectives. First, when data is stored in the blockchain, it lacks appropriate ways to specify the semantic relations as seen in relational database systems. It may cause data integrity problems and bring security risks. Second, ConsortiumSec is designed as a general-purpose framework for consortium applications. However, when it is applied to different applications, we still need to customize it for better performance. It requires approaches that can efficiently customize ConsortiumSec for different applications such as electronic medical record sharing and smart grid billing. Third, when ABAC policies are designed, there are tradeoffs between granularity and performance. Fine-grained policies are preferred for complicated consortium access control, but they can degrade the performance and scalability of the entire system. More research is needed to optimize the tradeoff.

References

- [1] Olivier Alphand, Michele Amoretti, Timothy Claeys, Simone Dall'Asta, Andrzej Duda, Gianluigi Ferrari, Franck Rousseau, Bernard Tourancheau, Luca Veltri, and Francesco Zanichelli. 2018. IoTChain: A blockchain security architecture for the Internet of Things. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1–6.
- [2] Benjamin Ampel, Mark Patton, and Hsinchun Chen. 2019. Performance modeling of hyperledger sawtooth blockchain. In *Proceedings of the IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, 59–61.
- [3] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, et al. 2018. Hyperledger fabric: A distributed operating system for permissioned blockchains. In *Proceedings of the 13th EuroSys Conference*, 1–15.
- [4] Carlo Argiolas, Nicoletta Dessi, Maria Grazia Fugini, and Barbara Pes. 2016. Enabling secure and collaborative document sharing in bim processes. In *Information Sciences and Systems 2015*. Springer, 393–402.
- [5] Farag Azzedin and Mustafa Ghaleb. 2019. Internet-of-Things and information fusion: Trust perspective survey. *Sensors* 19, 8 (2019), 1929.
- [6] Manas Bajaj, Dirk Zwemer, Russell Peak, Alex Phung, Andrew G. Scott, and Miyako Wilson. 2011. SLIM: Collaborative model-based systems engineering workspace for next-generation complex systems. In *Proceedings of the Aerospace Conference*, 1–15. DOI: <https://doi.org/10.1109/AERO.2011.5747539>
- [7] Arati Baliga, I. Subhod, Pandurang Kamat, and Siddhartha Chatterjee. 2018. Performance evaluation of the quorum blockchain platform. arXiv:1809.03421. Retrieved from <https://arxiv.org/abs/1809.03421>
- [8] Hugh Boyes. 2014. *Building Information Modelling (BIM): Addressing the Cyber Security Issues*. IET Cyber Security Consortium Report.
- [9] Sotirios Brotsis, Nicholas Kolokotronis, Konstantinos Limniotis, Gueltoum Bendiab, and Stavros Shiaeles. 2020. On the security and privacy of hyperledger fabric: Challenges and open issues. In *Proceedings of the IEEE World Congress on Services (SERVICES)*. IEEE, 197–204.
- [10] Richard Gendal Brown. 2018. The corda platform: An introduction. Retrieved 27 (2018), 2018.
- [11] Ahmed Youssef Bugalwi. 2020. *Blockchain-Based Trust Model: Alleviating the Threat of Malicious Cyber-attacks*. Ph.D. Dissertation. North Dakota State University.
- [12] Yaoliang Chen, Shi Chen, Jiao Liang, Lance Warren Feagan, Weili Han, Sheng Huang, and X. Sean Wang. 2020. Decentralized data access control over consortium blockchains. *Information Systems* 94 (2020), 101590.
- [13] Adam Conrad. 2021. Database of the year: Postgres. *IEEE Software* 38, 05 (2021), 130–132.
- [14] Chris Dannen. 2017. *Introducing Ethereum and Solidity*. Vol. 1. Springer.
- [15] Omar Dib, Kei-Leo Brousmiche, Antoine Durand, Eric Thea, and Elyes Ben Hamida. 2018. Consortium blockchains: Overview, applications and challenges. *International Journal on Advances in Telecommunications* 11, 1 & 2 (2018), 51–64.
- [16] Mingxiao Du, Qijun Chen, Jieying Chen, and Xiaofeng Ma. 2020. An optimized consortium blockchain for medical information sharing. *IEEE Transactions on Engineering Management* 68, 6 (2020), 1677–1689.

- [17] Yihai Fang and Xingliang Yuan. 2022. Security-aware data management in building information modeling processes using blockchain. In *Research Companion to Building Information Modeling*. Edward Elgar Publishing, 635–649.
- [18] Thippa Reddy Gadekallu, MK Manoj, Neeraj Kumar, Saqib Hakak, and Sweta Bhattacharya. 2021. Blockchain-based attack detection on machine learning algorithms for IoT-based e-health applications. *IEEE Internet of Things Magazine* 4, 3 (2021), 30–33.
- [19] Keke Gai, Yufeng She, Liehuang Zhu, Kim-Kwang Raymond Choo, and Zhiguo Wan. 2023. A blockchain-based access control scheme for zero trust cross-organizational data sharing. *ACM Transactions on Internet Technology* 23, 3 (2023), 1–25.
- [20] Oliver Gretscht, Frank Tietze, and Alexander Kock. 2020. Firms’ intellectual property ownership aggressiveness in university–industry collaboration projects: Choosing the right governance mode. *Creativity and Innovation Management* 29, 2 (2020), 359–370.
- [21] Ye Guo and Chen Liang. 2016. Blockchain application and outlook in the banking industry. *Financial Innovation* 2, 1 (2016), 1–12.
- [22] Vincent C. Hu, D. Richard Kuhn, David F. Ferraiolo, and Jeffrey Voas. 2015. Attribute-based access control. *Computer* 48, 2 (2015), 85–88.
- [23] Jayapriya Jayabalan and N. Jeyanthi. 2022. Scalable blockchain model using off-chain IPFS storage for healthcare data security and privacy. *J. Parallel and Distrib. Comput.* 164 (2022), 152–167.
- [24] Harry Kalodner, Malte Möser, Kevin Lee, Steven Goldfeder, Martin Plattner, Alishah Chator, and Arvind Narayanan. 2020. BlockSci: Design and applications of a blockchain analysis platform. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*. 2721–2738.
- [25] Shancang Li, Tao Qin, and Geyong Min. 2019. Blockchain-based digital forensics investigation framework in the internet of things and social systems. *IEEE Transactions on Computational Social Systems* 6, 6 (2019), 1433–1441.
- [26] Wei Liang, Yang Yang, Ce Yang, Yonghua Hu, Songyou Xie, Kuan-Ching Li, and Jiannong Cao. 2022. PDPChain: A consortium blockchain-based privacy protection scheme for personal data. *IEEE Transactions on Reliability* 72, 2 (2022), 586–598.
- [27] Chao Lin, Debiao He, Neeraj Kumar, Xinyi Huang, Pandi Vijayakumar, and Kim-Kwang Raymond Choo. 2019. HomeChain: A blockchain-based secure mutual authentication system for smart homes. *IEEE Internet of Things Journal* 7, 2 (2019), 818–829.
- [28] Iuon-Chang Lin and Tzu-Chun Liao. 2017. A survey of blockchain security issues and challenges. *International Journal of Network Security* 19, 5 (2017), 653–659.
- [29] Joshua Lind, Oded Naor, Ittay Eyal, Florian Kelbert, Peter Pietzuch, and Emin Gün Sirer. 2018. Teechain: Reducing storage costs on the blockchain with offline payment channels. In *Proceedings of the 11th ACM International Systems and Storage Conference*, 125–125.
- [30] Donatas Mazeika and Rimantas Butleris. 2020. Identifying security issues with mbse while rebuilding legacy software systems. In *Proceedings of the IEEE 15th International Conference on System of Systems Engineering (SoSE)*. IEEE, 83–86.
- [31] Donatas Mazeika and Rimantas Butleris. 2020. Integrating security requirements engineering into MBSE: Profile and guidelines. *Security and Communication Networks* 2020, 1 (2020), 5137625.
- [32] Daniel Migault, Marcos A. Simplicio, Bruno M. Barros, Makan Pourzandi, Thiago R. M. Almeida, Ewerton R. Andrade, and Tereza C. M. B. Carvalho. 2017. A framework for enabling security services collaboration across multiple domains. In *Proceedings of the IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 999–1010.
- [33] Satoshi Nakamoto. 2008. Bitcoin: A peer-to-peer electronic cash system. *Bitcoin* 4, 2 (2008). Retrieved from <https://bitcoin.org/bitcoin.pdf>
- [34] Michele Nitti, Roberto Girau, and Luigi Atzori. 2013. Trustworthiness management in the social internet of things. *IEEE Transactions on Knowledge and Data Engineering* 26, 5 (2013), 1253–1266.
- [35] Oscar Novo. 2018. Blockchain meets IoT: An architecture for scalable access management in IoT. *IEEE Internet of Things Journal* 5, 2 (2018), 1184–1195.
- [36] OASIS. 2015. *Abbreviated Language for Authorization (ALFA) Version 1.0*.
- [37] Niklas Palaghias, Nikos Loumis, Stylianos Georgoulas, and Klaus Moessner. 2016. Quantifying trust relationships based on real-world social interactions. In *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 1–7.
- [38] Deeptiman Pattnaik. 2020. *BlockReader Application*. GitHub, Inc.
- [39] Amar Rasheed, Rabi N Mahapatra, Cihan Varol, and Karpoor Narashimha. 2021. Exploiting zero knowledge proof and blockchains towards the enforcement of anonymity, data integrity and privacy (ADIP) in the IoT. *IEEE Transactions on Emerging Topics in Computing* 10, 3 (2021), 1476–1491.
- [40] Andrea M Rozario and Chanta Thomas. 2019. Reengineering the audit with blockchain and smart contracts. *Journal of Emerging Technologies in Accounting* 16, 1 (2019), 21–35.
- [41] OASIS Standard. 2013. Extensible Access Control Markup Language (xacml) Version 3.0. Retrieved from <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>
- [42] Nadia Tagoe, Sassy Molyneux, Justin Pulford, and Sam Kinyanjui. 2022. Consortium management structures, processes, and approaches: The DELTAS Africa example. *Wellcome Open Research* 7, 139 (2022), 139.
- [43] Bruno Tavares, F. Figueiredo Correia, and André Restivo. 2019. A survey on blockchain technologies and research. *Journal of Information Assurance and Security* 14 (2019), 118–128.
- [44] Yan Wang and Zenan Wu. 2023. Blockchain-based multidimensional trust management in edge computing. *IEEE Access* (2023).
- [45] Kwok-Bun Yue, Mark Guerra, Howard Wagner, Joses Sandeep Thamara Selvan, Kayaanoosh Collector, Victor Tang, Margo Sikes, Kewei Sha, Pavani Kallempudi, Sahar Mardani, et al. 2021. Applying blockchain technology on model-based systems engineering. In *Proceedings of the International Conference on AIAA Scitech 2021 Forum*, 0093.

- [46] Kwok-Bun Yue, Kewei Sha, Jose Sandeep Thamarai Selvan, Mark Guerra, Wei Wei, Shanmuksai Chakka, Preethi Vuchuru, Madhuri Koduru, Xinying Liu, Victor Tang, et al. 2023. Confidentiality and data integrity in consortium blockchain applications for model-based systems engineering. In *Proceedings of the International Conference on ALAA Scitech 2023 Forum*.
- [47] Riccardo Zappoli. 2022. *Go Language Support in Hyperledger Fabric Private Chaincode*. Master Thesis. Université de Fribourg.
- [48] Yinghui Zhang, Robert H Deng, Shengmin Xu, Jianfei Sun, Qi Li, and Dong Zheng. 2020. Attribute-based encryption for cloud computing access control: A survey. *ACM Computing Surveys* 53, 4 (2020), 1–41.
- [49] Botao Zhong, Haitao Wu, Lieyun Ding, Hanbin Luo, Ying Luo, and Xing Pan. 2020. Hyperledger fabric-based consortium blockchain for construction quality information management. *Frontiers of Engineering Management* 7, 4 (2020), 512–527.
- [50] Ence Zhou, Haoli Sun, Bingfeng Pi, Jun Sun, Kazuhiro Yamashita, and Yoshihide Nomura. 2019. Ledgerdata refiner: A powerful ledger data query platform for hyperledger fabric. In *Proceedings of the 6th International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE, 433–440.

Received 25 October 2023; revised 28 August 2024; accepted 25 September 2024